

End-to-End Pipeline — Orchestration & Validation

ServiceTrack Project | Execution Screenshots

Screenshot 1: Validate Row Counts Across Layers



```
1 # Load Delta tables to validate
2 bronze_count = spark.read.table("bronze_service_jobs").count()
3 silver_count = spark.read.table("silver_enriched_jobs").count()
4
5 print("-----")
6 print(f"Raw Ingested Records (Bronze): {bronze_count}")
7 print(f"Clean Enriched Records (Silver): {silver_count}")
8 print("-----")
9
10 # Simple validation checks
11 if bronze_count == 1510 and silver_count == 1500:
12     print("Success: Row counts matched expected values (10 duplicate records successfully removed).")
13 else:
14     print("Warning: Row counts deviated from standard test records.")
15
16 > See performance \(2\) ✓ Checked for improvements
```

Raw Ingested Records (Bronze): 1510
Clean Enriched Records (Silver): 1500

Success: Row counts matched expected values (10 duplicate records successfully removed).

Row count validation confirms Bronze (1510) to Silver (1500) deduplication succeeded — 10 duplicate records removed.

Screenshot 2: Validate technician_name Has No Nulls

Validate: technician_name Has No Nulls

```
1 # Check that technician_name has zero nulls in the Silver table
2 null_tech_count = spark.read.table("silver_enriched_jobs").filter(col("technician_name").isNull()).count()
3
4 print("-----")
5 print(f"Null technician_name count in Silver: {null_tech_count}")
6 print("-----")
7
8 if null_tech_count == 0:
9     print("Success: No null technician_name values found.")
10 else:
11     print("Warning: Some technician_name values are still null. Review the Silver layer lookup logic.")
```

> [See performance \(1\)](#) ✓ Checked for improvements

```
-----
Null technician_name count in Silver: 0
-----
Success: No null technician_name values found.
```

Validation confirms zero null values remain in the technician_name column of the Silver table.

Screenshot 3: Validate No Negative Costs

Validate: No Negative Costs

▶

✓ Just now (1s)

16

Python

🗑️

🔗

📄

⋮

```
1 # Check for negative costs
2 df_silver_check = spark.read.table(SILVER_ENRICHED_JOBS_TABLE)
3 negative_cost_count = df_silver_check.filter(
4     (col("estimated_cost") < 0) | (col("actual_cost") < 0)
5 ).count()
6
7 if negative_cost_count == 0:
8     logger.info("Success: No negative costs found.")
9 else:
10    logger.warning(f"Warning: {negative_cost_count} rows have negative costs. Review the data
    quality rules.")
```

> df_silver_check: pyspark.sql.connect.dataframe.DataFrame = [device_id: string, customer_id: string ... 23 more fields]

2026-07-11 20:25:14,334 - INFO - Success: No negative costs found.

Validation confirms no rows have negative `estimated_cost` or `actual_cost` values in the Silver table — a new data quality check added to the pipeline.

Screenshot 4: Print Overall Pipeline Success Message



The screenshot shows a code editor with a dark theme. At the top, the title 'Print Overall Pipeline Success Message' is displayed. Below the title, there is a code editor window. The code editor has a toolbar at the top with a play button, a dropdown menu, a status indicator 'Just now (<1s)', a line number '20', a language selector 'Python', and icons for trash, copy, and zoom. The code is as follows:

```
1 logger.info("End-to-end pipeline validation complete.")
2 print("=====")
3 print("  END-TO-END PIPELINE COMPLETED SUCCESSFULLY!  ")
4 print("=====")
```

Below the code, the output is shown. It starts with a timestamp and log level: '2026-07-11 20:29:14,177 - INFO - End-to-end pipeline validation complete.'. This is followed by a series of equals signs, the success message 'END-TO-END PIPELINE COMPLETED SUCCESSFULLY!', and another series of equals signs.

Final confirmation message printed, indicating the end-to-end pipeline completed successfully.